

Bayesian Logistic Regression

Ella Chee

2026-02-27

Bayesian Model for Wildfire Detection

Get data from Kaggle

```
library(kagglr)

# Authenticate creds
#kgl_auth(username = "ellachee21", key = "KGAT_107e527829b7a3f5505e98b50142d7db")

# Download a dataset
# system("/Users/nutellachee/anaconda3/bin/kaggle datasets download -d abdelghaniaaba/wildfire-predicti
```

Load data

```
library(dplyr)

##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##   filter, lag

## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union

library(tidyverse)

## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v forcats   1.0.1    v readr     2.2.0
## v ggplot2   4.0.1    v stringr  1.6.0
## v lubridate 1.9.4    v tibble   3.3.1
## v purrr     1.2.0    v tidyr    1.3.2
```

```
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag() masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors

# Helper to load both classes (no wildfire/wildfire) for given folder
load_data <- function(folder_name) {

  nowildfire_list <- list.files(paste0("data/", folder_name, "/nowildfire"), full.names = TRUE)
  wildfire_list <- list.files(paste0("data/", folder_name, "/wildfire"), full.names = TRUE)

  print(sprintf("%d images in no wildfire class", length(nowildfire_list)))
  print(sprintf("%d images in wildfire class", length(wildfire_list)))

  return(list(nowildfire_list, wildfire_list))

}
```

Load validation, training, testing sets

```
# Load validation data
print("VALIDATION:")
```

```
## [1] "VALIDATION:"
```

```
valid_results <- load_data("valid")
```

```
## [1] "2820 images in no wildfire class"
## [1] "3480 images in wildfire class"
```

```
valid_nowildfire <- valid_results[[1]]
valid_wildfire <- valid_results[[2]]
```

```
# Load training data
print("TRAINING:")
```

```
## [1] "TRAINING:"
```

```
train_results <- load_data("train")
```

```
## [1] "14499 images in no wildfire class"
## [1] "15750 images in wildfire class"
```

```
train_nowildfire <- train_results[[1]]
train_wildfire <- train_results[[2]]
```

```
# Load testing data
print("TESTING:")
```

```
## [1] "TESTING:"
```

```
test_results <- load_data("test")

## [1] "2820 images in no wildfire class"
## [1] "3479 images in wildfire class"

test_nowildfire <- test_results[[1]]
test_wildfire <- test_results[[2]]
```

Preview image data

```
library(magick)

## Linking to ImageMagick 6.9.13.29
## Enabled features: cairo, fontconfig, freetype, heic, lcms, pango, raw, rsvg, webp
## Disabled features: fftw, ghostscript, x11

# Randomly select image
set.seed(42)
random_img <- sample(test_wildfire, 1)

# Load image into R
img <- image_read(random_img)
print(img)

## # A tibble: 1 x 7
##   format width height colorspace matte filesize density
##   <chr>   <int>  <int> <chr>      <lgl>    <int> <chr>
## 1 JPEG     350    350 sRGB      FALSE    25099 72x72
```



```
# Load image as array of values
img_array <- as.numeric(image_data(img, channels = "rgb"))
dim(img_array)
```

```
## [1] 350 350 3
```

Data preparation

```
library(magick)
library(dplyr)

# Using each pixel as a separate feature
load_images_to_df <- function(image_paths, label, batch_size = 500, resize = 35) {

  # Batch to increase efficiency
  batches <- split(image_paths, ceiling(seq_along(image_paths) / batch_size))
  results <- lapply(batches, function(batch) {

    imgs <- tryCatch(
      image_read(batch) |>
      image_resize(paste0(resize, "x", resize, "!")),
```

```

    error = function(e) { message("Batch error: ", e$message); NULL }
  )
  if (is.null(imgs)) return(NULL)

  n <- length(batch)
  p <- resize * resize

  # Pre-allocate matrix
  X <- matrix(NA_real_, nrow = n, ncol = p)

  for (i in seq_len(n)) {
    arr <- image_data(imgs[i], channels = "rgb") # [channel, x, y]

    R <- as.integer(arr[1,,]) # Extract separate channels
    G <- as.integer(arr[2,,])
    B <- as.integer(arr[3,,])

    pixel_vals <- R * 256^2 + G * 256 + B # Combine into single value per pixel
    X[i, ] <- as.numeric(pixel_vals) # Flatten to vector
  }
  image_destroy(imgs)

  # Convert to dataframe
  df <- as.data.frame(X)
  colnames(df) <- paste0("px_", seq_len(p))
  #df$name <- batch
  df$label <- label
  df
})
bind_rows(results)
}

# Load each dataset
dfs <- list()

for (name in c("train", "valid", "test")) {
  nowildfire_df <- load_images_to_df(get(paste0(name, "_nowildfire")), 0, 500, 16)
  wildfire_df <- load_images_to_df(get(paste0(name, "_wildfire")), 1, 500, 16)

  # Combine and shuffle
  df <- bind_rows(nowildfire_df, wildfire_df)
  set.seed(42)
  shuffled_df <- df[sample(1:nrow(df)), ]

  dfs[[name]] <- shuffled_df
  print(sprintf("Loaded %s dataset", name))
}

```

```

## [1] "Loaded train dataset"
## [1] "Loaded valid dataset"
## [1] "Loaded test dataset"

```

```
# Apply to each dataset
train_df <- dfs[["train"]]
valid_df <- dfs[["valid"]]
test_df <- dfs[["test"]]
```

Model set-up

1. Resized 350x350px to 35x35px -> flattened to 1225 px values = 1225 features, with chains = 4, iter = 20, warmup = 3, 68 divergent chains after warmup, largest R-hat is Inf, indicating chains have not mixed, vector memory limit of 24.0 GB reached
2. Resized to 16x16 px -> flattened to 256 px values = 256 features, with chains = 4, iter = 100, warmup = 18, 327 divergent transitions after warmup, largest R-hat is 30.33, indicating chains have not mixed vector memory limit of 24.0 Gb reached, see mem.maxVSize()
3. Resized to 16x16 px -> flattened to 256 px values = 256 features, with chains = 4, iter = 500, warmup = 90, 616 divergent transitions after warmup, largest R-hat is R-hat is 7.38, indicating chains have not mixed vector memory limit of 24.0 Gb reached, see mem.maxVSize()
4. Resized to 16x16 px -> flattened to 256 px values = 256 features, with chains = 4, iter = 1000, warmup = 180, 199 divergent transitions after warmup, largest R-hat is R-hat is 4.75, indicating chains have not mixed
5. Resized to 16x16 px -> flattened to 256 px values = 256 features, with chains = 6, iter = 10000, warmup = 1800, 2540 divergent transitions after warmup, largest R-hat is R-hat is 5.48, indicating chains have not mixed

```
library(brms)
```

```
## Loading required package: Rcpp
```

```
## Loading 'brms' package (version 2.23.0). Useful instructions
## can be found by typing help('brms'). A more detailed introduction
## to the package is available through vignette('brms_overview').
```

```
##
```

```
## Attaching package: 'brms'
```

```
## The following object is masked from 'package:stats':
```

```
##
```

```
##      ar
```

```
# Set default prior (use Bernoulli for binary classification)
```

```
default_prior <- default_prior(label ~ .,
                                data = train_df,
                                family = bernoulli("logit"))
```

```
# Creating Bayesian logistic regression model, use all pixels as features
```

```
brm_logit <- brm(label ~ .,
                 family = bernoulli("logit"),
                 data = train_df,
                 chains = 6,
```

```

        iter = 10000,
        warmup = 1800,
        prior = default_prior)

## Compiling Stan program...

## Trying to compile a simple C file

## Running /Library/Frameworks/R.framework/Resources/bin/R CMD SHLIB foo.c
## using C compiler: 'Apple clang version 15.0.0 (clang-1500.1.0.2.5)'
## using SDK: 'MacOSX14.2.sdk'
## clang -arch arm64 -std=gnu2x -I"/Library/Frameworks/R.framework/Resources/include" -DNDEBUG -I"/Li
## In file included from <built-in>:1:
## In file included from /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/library/StanHeader
## In file included from /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/library/RcppEigen
## In file included from /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/library/RcppEigen
## /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/library/RcppEigen/include/Eigen/src/Core
## #include <cmath>
##      ~~~~~
## 1 error generated.
## make: *** [foo.o] Error 1

## Start sampling

##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
## Chain 1:
## Chain 1: Gradient evaluation took 0.002033 seconds
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 20.33 seconds.
## Chain 1: Adjust your expectations accordingly!
## Chain 1:
## Chain 1:
## Chain 1: Iteration:    1 / 10000 [ 0%] (Warmup)
## Chain 1: Iteration: 1000 / 10000 [ 10%] (Warmup)
## Chain 1: Iteration: 1801 / 10000 [ 18%] (Sampling)
## Chain 1: Iteration: 2800 / 10000 [ 28%] (Sampling)
## Chain 1: Iteration: 3800 / 10000 [ 38%] (Sampling)
## Chain 1: Iteration: 4800 / 10000 [ 48%] (Sampling)
## Chain 1: Iteration: 5800 / 10000 [ 58%] (Sampling)
## Chain 1: Iteration: 6800 / 10000 [ 68%] (Sampling)
## Chain 1: Iteration: 7800 / 10000 [ 78%] (Sampling)
## Chain 1: Iteration: 8800 / 10000 [ 88%] (Sampling)
## Chain 1: Iteration: 9800 / 10000 [ 98%] (Sampling)
## Chain 1: Iteration: 10000 / 10000 [100%] (Sampling)
## Chain 1:
## Chain 1: Elapsed Time: 33.574 seconds (Warm-up)
## Chain 1:                    55.683 seconds (Sampling)
## Chain 1:                    89.257 seconds (Total)
## Chain 1:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).
## Chain 2:

```

```

## Chain 2: Gradient evaluation took 0.002744 seconds
## Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 27.44 seconds.
## Chain 2: Adjust your expectations accordingly!
## Chain 2:
## Chain 2:
## Chain 2: Iteration:      1 / 10000 [ 0%] (Warmup)
## Chain 2: Iteration: 1000 / 10000 [10%] (Warmup)
## Chain 2: Iteration: 1801 / 10000 [18%] (Sampling)
## Chain 2: Iteration: 2800 / 10000 [28%] (Sampling)
## Chain 2: Iteration: 3800 / 10000 [38%] (Sampling)
## Chain 2: Iteration: 4800 / 10000 [48%] (Sampling)
## Chain 2: Iteration: 5800 / 10000 [58%] (Sampling)
## Chain 2: Iteration: 6800 / 10000 [68%] (Sampling)
## Chain 2: Iteration: 7800 / 10000 [78%] (Sampling)
## Chain 2: Iteration: 8800 / 10000 [88%] (Sampling)
## Chain 2: Iteration: 9800 / 10000 [98%] (Sampling)
## Chain 2: Iteration: 10000 / 10000 [100%] (Sampling)
## Chain 2:
## Chain 2: Elapsed Time: 34.827 seconds (Warm-up)
## Chain 2:                75.129 seconds (Sampling)
## Chain 2:                109.956 seconds (Total)
## Chain 2:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).
## Chain 3:
## Chain 3: Gradient evaluation took 0.001952 seconds
## Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 19.52 seconds.
## Chain 3: Adjust your expectations accordingly!
## Chain 3:
## Chain 3:
## Chain 3: Iteration:      1 / 10000 [ 0%] (Warmup)
## Chain 3: Iteration: 1000 / 10000 [10%] (Warmup)
## Chain 3: Iteration: 1801 / 10000 [18%] (Sampling)
## Chain 3: Iteration: 2800 / 10000 [28%] (Sampling)
## Chain 3: Iteration: 3800 / 10000 [38%] (Sampling)
## Chain 3: Iteration: 4800 / 10000 [48%] (Sampling)
## Chain 3: Iteration: 5800 / 10000 [58%] (Sampling)
## Chain 3: Iteration: 6800 / 10000 [68%] (Sampling)
## Chain 3: Iteration: 7800 / 10000 [78%] (Sampling)
## Chain 3: Iteration: 8800 / 10000 [88%] (Sampling)
## Chain 3: Iteration: 9800 / 10000 [98%] (Sampling)
## Chain 3: Iteration: 10000 / 10000 [100%] (Sampling)
## Chain 3:
## Chain 3: Elapsed Time: 33.475 seconds (Warm-up)
## Chain 3:                51.405 seconds (Sampling)
## Chain 3:                84.88 seconds (Total)
## Chain 3:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).
## Chain 4:
## Chain 4: Gradient evaluation took 0.001997 seconds
## Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 19.97 seconds.
## Chain 4: Adjust your expectations accordingly!
## Chain 4:

```



```

## Chain 4:
## Chain 4: Iteration:      1 / 10000 [ 0%] (Warmup)
## Chain 4: Iteration: 1000 / 10000 [ 10%] (Warmup)
## Chain 4: Iteration: 1801 / 10000 [ 18%] (Sampling)
## Chain 4: Iteration: 2800 / 10000 [ 28%] (Sampling)
## Chain 4: Iteration: 3800 / 10000 [ 38%] (Sampling)
## Chain 4: Iteration: 4800 / 10000 [ 48%] (Sampling)
## Chain 4: Iteration: 5800 / 10000 [ 58%] (Sampling)
## Chain 4: Iteration: 6800 / 10000 [ 68%] (Sampling)
## Chain 4: Iteration: 7800 / 10000 [ 78%] (Sampling)
## Chain 4: Iteration: 8800 / 10000 [ 88%] (Sampling)
## Chain 4: Iteration: 9800 / 10000 [ 98%] (Sampling)
## Chain 4: Iteration: 10000 / 10000 [100%] (Sampling)
## Chain 4:
## Chain 4: Elapsed Time: 35.516 seconds (Warm-up)
## Chain 4:                157.065 seconds (Sampling)
## Chain 4:                192.581 seconds (Total)
## Chain 4:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 5).
## Chain 5:
## Chain 5: Gradient evaluation took 0.002032 seconds
## Chain 5: 1000 transitions using 10 leapfrog steps per transition would take 20.32 seconds.
## Chain 5: Adjust your expectations accordingly!
## Chain 5:
## Chain 5:
## Chain 5: Iteration:      1 / 10000 [ 0%] (Warmup)
## Chain 5: Iteration: 1000 / 10000 [ 10%] (Warmup)
## Chain 5: Iteration: 1801 / 10000 [ 18%] (Sampling)
## Chain 5: Iteration: 2800 / 10000 [ 28%] (Sampling)
## Chain 5: Iteration: 3800 / 10000 [ 38%] (Sampling)
## Chain 5: Iteration: 4800 / 10000 [ 48%] (Sampling)
## Chain 5: Iteration: 5800 / 10000 [ 58%] (Sampling)
## Chain 5: Iteration: 6800 / 10000 [ 68%] (Sampling)
## Chain 5: Iteration: 7800 / 10000 [ 78%] (Sampling)
## Chain 5: Iteration: 8800 / 10000 [ 88%] (Sampling)
## Chain 5: Iteration: 9800 / 10000 [ 98%] (Sampling)
## Chain 5: Iteration: 10000 / 10000 [100%] (Sampling)
## Chain 5:
## Chain 5: Elapsed Time: 27.772 seconds (Warm-up)
## Chain 5:                45.955 seconds (Sampling)
## Chain 5:                73.727 seconds (Total)
## Chain 5:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 6).
## Chain 6:
## Chain 6: Gradient evaluation took 0.002109 seconds
## Chain 6: 1000 transitions using 10 leapfrog steps per transition would take 21.09 seconds.
## Chain 6: Adjust your expectations accordingly!
## Chain 6:
## Chain 6:
## Chain 6: Iteration:      1 / 10000 [ 0%] (Warmup)
## Chain 6: Iteration: 1000 / 10000 [ 10%] (Warmup)
## Chain 6: Iteration: 1801 / 10000 [ 18%] (Sampling)

```

```
## Chain 6: Iteration: 2800 / 10000 [ 28%] (Sampling)
## Chain 6: Iteration: 3800 / 10000 [ 38%] (Sampling)
## Chain 6: Iteration: 4800 / 10000 [ 48%] (Sampling)
## Chain 6: Iteration: 5800 / 10000 [ 58%] (Sampling)
## Chain 6: Iteration: 6800 / 10000 [ 68%] (Sampling)
## Chain 6: Iteration: 7800 / 10000 [ 78%] (Sampling)
## Chain 6: Iteration: 8800 / 10000 [ 88%] (Sampling)
## Chain 6: Iteration: 9800 / 10000 [ 98%] (Sampling)
## Chain 6: Iteration: 10000 / 10000 [100%] (Sampling)
## Chain 6:
## Chain 6: Elapsed Time: 33.551 seconds (Warm-up)
## Chain 6: 174.594 seconds (Sampling)
## Chain 6: 208.145 seconds (Total)
## Chain 6:
```

```
## Warning: There were 20258 divergent transitions after warmup. See
## https://mc-stan.org/misc/warnings.html#divergent-transitions-after-warmup
## to find out why this is a problem and how to eliminate them.
```

```
## Warning: There were 6 chains where the estimated Bayesian Fraction of Missing Information was low. See
## https://mc-stan.org/misc/warnings.html#bfmi-low
```

```
## Warning: Examine the pairs() plot to diagnose sampling problems
```

```
## Warning: The largest R-hat is 7.52, indicating chains have not mixed.
## Running the chains for more iterations may help. See
## https://mc-stan.org/misc/warnings.html#r-hat
```

```
## Warning: Bulk Effective Samples Size (ESS) is too low, indicating posterior means and medians may be
## Running the chains for more iterations may help. See
## https://mc-stan.org/misc/warnings.html#bulk-ess
```

```
## Warning: Tail Effective Samples Size (ESS) is too low, indicating posterior variances and tail quantiles
## Running the chains for more iterations may help. See
## https://mc-stan.org/misc/warnings.html#tail-ess
```

```
# Get summary stats on posterior
summary(brm_logit)
```

```
## Warning: Parts of the model have not converged (some Rhats are > 1.05). Be
## careful when analysing the results! We recommend running more iterations and/or
## setting stronger priors.
```

```
## Warning: There were 20258 divergent transitions after warmup. Increasing
## adapt_delta above 0.8 may help. See
## http://mc-stan.org/misc/warnings.html#divergent-transitions-after-warmup
```

```
## Family: bernoulli
## Links: mu = logit
## Formula: label ~ px_1 + px_2 + px_3 + px_4 + px_5 + px_6 + px_7 + px_8 + px_9 + px_10 + px_11 + px_12
## Data: train_df (Number of observations: 30249)
```

```

## Draws: 6 chains, each with iter = 10000; warmup = 1800; thin = 1;
## total post-warmup draws = 49200
##
## Regression Coefficients:
##      Estimate Est.Error  l-95% CI  u-95% CI Rhat Bulk_ESS Tail_ESS
## Intercept 3771443.66 2140362.31 1153468.37 7593039.24 4.79      6      6
## px_1      -0.09      0.06      -0.25      -0.02 5.29      6     11
## px_2      -0.04      0.15      -0.34      0.21 5.67      6      6
## px_3       0.04      0.12      -0.16      0.22 5.62      6     11
## px_4      -0.05      0.12      -0.28      0.13 5.77      6      6
## px_5       0.04      0.11      -0.16      0.22 5.40      6     11
## px_6      -0.07      0.11      -0.24      0.13 5.20      6      6
## px_7       0.07      0.11      -0.07      0.26 5.56      6     11
## px_8      -0.05      0.10      -0.20      0.11 4.89      6      6
## px_9       0.01      0.14      -0.28      0.11 3.76      7      6
## px_10     -0.00      0.10      -0.20      0.18 6.84      6     11
## px_11      0.02      0.07      -0.12      0.15 5.16      6      6
## px_12     -0.09      0.06      -0.24     -0.01 3.34      7     11
## px_13     -0.00      0.08      -0.15      0.14 4.76      6      6
## px_14     -0.04      0.14      -0.29      0.13 5.47      6      6
## px_15      0.04      0.18      -0.18      0.32 3.82      7      6
## px_16     -0.07      0.13      -0.33      0.07 5.29      6      6
## px_17      0.06      0.11      -0.07      0.28 6.16      6      6
## px_18      0.06      0.25      -0.33      0.49 5.29      6     11
## px_19     -0.06      0.20      -0.32      0.26 5.72      6      6
## px_20      0.06      0.20      -0.28      0.39 5.54      6     11
## px_21     -0.07      0.21      -0.37      0.32 5.26      6      6
## px_22      0.07      0.15      -0.22      0.26 5.50      6     11
## px_23     -0.11      0.17      -0.38      0.06 5.23      7      6
## px_24      0.06      0.15      -0.22      0.28 5.30      6      6
## px_25      0.02      0.27      -0.16      0.60 4.24      6     22
## px_26     -0.08      0.23      -0.56      0.15 5.66      6      6
## px_27      0.06      0.18      -0.09      0.45 5.55      6     11
## px_28      0.02      0.02      -0.01      0.09 2.36      7      6
## px_29      0.05      0.11      -0.05      0.25 4.79      6     11
## px_30      0.02      0.12      -0.16      0.22 5.28      6      6
## px_31     -0.02      0.14      -0.23      0.16 4.88      6      6
## px_32      0.01      0.10      -0.15      0.18 6.95      6     NA
## px_33     -0.07      0.07      -0.21      0.02 5.88      6     NA
## px_34     -0.08      0.17      -0.31      0.22 5.07      6      6
## px_35      0.11      0.20      -0.16      0.50 5.02      6     11
## px_36     -0.10      0.23      -0.49      0.25 6.62      6      6
## px_37      0.08      0.23      -0.27      0.45 5.40      6     11
## px_38     -0.00      0.13      -0.20      0.21 7.26      6      6
## px_39      0.02      0.20      -0.19      0.43 5.34      6     11
## px_40     -0.00      0.08      -0.09      0.13 4.93      6     11
## px_41     -0.05      0.19      -0.46      0.10 5.58      6      6
## px_42      0.09      0.17      -0.08      0.44 3.55      7      6
## px_43     -0.10      0.20      -0.51      0.12 5.03      6      6
## px_44      0.01      0.10      -0.18      0.13 4.98      6      6
## px_45     -0.03      0.06      -0.13      0.04 3.39      7      6
## px_46     -0.01      0.07      -0.13      0.08 4.53      6     11
## px_47     -0.01      0.08      -0.13      0.09 4.56      6     NA
## px_48     -0.01      0.09      -0.19      0.11 5.96      6      6

```

## px_49	0.06	0.04	0.01	0.15 5.16	6	11
## px_50	0.00	0.09	-0.17	0.15 5.83	6	11
## px_51	-0.02	0.13	-0.36	0.14 5.45	6	11
## px_52	0.05	0.14	-0.14	0.28 3.40	7	6
## px_53	-0.07	0.19	-0.41	0.14 4.76	6	6
## px_54	0.01	0.11	-0.15	0.19 3.73	6	13
## px_55	-0.06	0.15	-0.30	0.09 5.26	6	11
## px_56	0.02	0.16	-0.24	0.39 5.20	6	6
## px_57	0.01	0.26	-0.44	0.51 5.65	6	11
## px_58	0.00	0.19	-0.34	0.29 5.70	6	6
## px_59	-0.00	0.12	-0.12	0.23 4.97	6	6
## px_60	0.08	0.13	-0.13	0.29 4.39	6	16
## px_61	-0.03	0.05	-0.08	0.08 3.13	7	6
## px_62	0.05	0.12	-0.06	0.29 4.58	6	23
## px_63	-0.05	0.16	-0.31	0.13 4.71	6	6
## px_64	0.01	0.13	-0.23	0.24 3.59	7	11
## px_65	-0.10	0.11	-0.35	-0.00 4.98	6	6
## px_66	0.03	0.10	-0.09	0.24 5.86	6	11
## px_67	-0.01	0.15	-0.29	0.32 4.16	6	6
## px_68	-0.05	0.13	-0.30	0.15 4.47	6	11
## px_69	0.07	0.13	-0.11	0.38 5.47	6	6
## px_70	-0.02	0.09	-0.19	0.08 4.70	6	NA
## px_71	0.10	0.11	-0.02	0.31 4.83	6	11
## px_72	-0.04	0.15	-0.36	0.22 4.32	6	11
## px_73	-0.04	0.23	-0.50	0.24 5.12	6	6
## px_74	-0.04	0.12	-0.33	0.13 5.39	6	11
## px_75	0.04	0.05	-0.03	0.12 3.59	7	12
## px_76	-0.07	0.11	-0.29	0.05 5.22	6	6
## px_77	-0.00	0.08	-0.13	0.11 3.83	6	6
## px_78	-0.07	0.10	-0.35	0.03 4.29	6	11
## px_79	0.07	0.12	-0.10	0.28 4.96	6	11
## px_80	-0.00	0.08	-0.12	0.15 5.06	6	6
## px_81	0.06	0.08	-0.05	0.22 4.61	6	11
## px_82	-0.07	0.10	-0.29	-0.00 2.78	7	6
## px_83	0.05	0.17	-0.27	0.38 5.06	6	11
## px_84	0.04	0.11	-0.09	0.28 6.85	6	7
## px_85	-0.06	0.17	-0.35	0.12 4.12	6	11
## px_86	-0.04	0.10	-0.16	0.21 3.85	6	11
## px_87	-0.03	0.14	-0.25	0.19 4.94	6	11
## px_88	-0.01	0.14	-0.18	0.33 5.16	6	6
## px_89	0.06	0.15	-0.18	0.33 6.76	6	NA
## px_90	0.07	0.14	-0.11	0.38 5.90	6	6
## px_91	-0.03	0.08	-0.17	0.09 4.36	6	16
## px_92	0.05	0.15	-0.20	0.25 4.92	7	6
## px_93	0.01	0.13	-0.18	0.26 5.00	6	11
## px_94	0.04	0.13	-0.18	0.33 4.70	6	6
## px_95	-0.04	0.07	-0.15	0.08 4.38	6	11
## px_96	-0.01	0.07	-0.14	0.09 5.80	6	11
## px_97	-0.02	0.06	-0.12	0.09 3.64	7	7
## px_98	0.04	0.08	-0.03	0.20 5.14	6	6
## px_99	0.01	0.11	-0.15	0.30 5.01	6	6
## px_100	-0.11	0.13	-0.34	0.03 4.40	6	NA
## px_101	0.05	0.18	-0.17	0.36 4.51	6	6
## px_102	0.06	0.11	-0.18	0.17 5.24	6	11

## px_103	-0.01	0.12	-0.19	0.18 4.49	6	11
## px_104	0.03	0.13	-0.15	0.25 5.59	6	11
## px_105	-0.11	0.15	-0.43	0.06 3.15	7	6
## px_106	-0.03	0.08	-0.18	0.08 3.95	6	6
## px_107	-0.01	0.11	-0.39	0.08 3.36	7	11
## px_108	-0.02	0.12	-0.23	0.28 4.72	6	6
## px_109	-0.01	0.07	-0.18	0.06 3.56	7	11
## px_110	-0.05	0.07	-0.25	0.04 4.04	6	11
## px_111	0.05	0.06	-0.05	0.16 4.11	6	6
## px_112	-0.02	0.06	-0.13	0.06 4.95	6	6
## px_113	0.02	0.06	-0.07	0.12 4.09	6	11
## px_114	-0.09	0.14	-0.34	0.05 4.98	6	6
## px_115	0.04	0.16	-0.20	0.34 5.43	6	11
## px_116	0.06	0.16	-0.13	0.40 5.50	6	6
## px_117	-0.00	0.15	-0.31	0.20 5.39	6	11
## px_118	-0.05	0.17	-0.33	0.27 4.32	6	6
## px_119	-0.02	0.17	-0.36	0.15 3.78	7	11
## px_120	-0.02	0.18	-0.31	0.24 5.09	6	6
## px_121	0.08	0.18	-0.16	0.44 5.12	6	6
## px_122	-0.00	0.07	-0.14	0.11 5.83	6	7
## px_123	0.01	0.07	-0.12	0.13 4.65	6	6
## px_124	0.03	0.10	-0.10	0.23 4.97	6	25
## px_125	-0.06	0.11	-0.28	0.07 4.67	6	6
## px_126	0.10	0.09	-0.04	0.26 4.16	6	6
## px_127	-0.09	0.11	-0.26	0.11 4.80	6	6
## px_128	0.04	0.12	-0.13	0.24 5.78	6	11
## px_129	-0.09	0.09	-0.22	0.04 3.35	7	6
## px_130	0.14	0.16	-0.06	0.43 4.04	6	6
## px_131	-0.08	0.19	-0.37	0.22 5.11	6	6
## px_132	0.02	0.21	-0.49	0.24 5.46	6	11
## px_133	-0.05	0.20	-0.25	0.43 6.31	6	6
## px_134	0.01	0.19	-0.44	0.17 6.65	6	11
## px_135	0.07	0.23	-0.20	0.64 4.20	6	11
## px_136	-0.07	0.21	-0.63	0.21 5.50	6	11
## px_137	0.04	0.15	-0.19	0.30 5.14	6	7
## px_138	-0.07	0.16	-0.48	0.17 5.06	6	11
## px_139	0.07	0.14	-0.13	0.32 5.62	6	7
## px_140	-0.08	0.17	-0.43	0.08 5.04	6	6
## px_141	0.12	0.15	-0.02	0.44 5.71	6	7
## px_142	-0.16	0.12	-0.40	-0.05 6.50	6	NA
## px_143	0.10	0.17	-0.26	0.32 5.42	6	11
## px_144	-0.04	0.13	-0.22	0.16 5.30	6	6
## px_145	0.07	0.07	-0.04	0.15 4.27	6	6
## px_146	-0.08	0.08	-0.18	0.06 4.33	6	21
## px_147	-0.02	0.15	-0.37	0.13 6.53	6	11
## px_148	0.06	0.22	-0.17	0.71 6.14	6	6
## px_149	0.04	0.21	-0.55	0.23 6.05	6	11
## px_150	-0.02	0.17	-0.16	0.41 5.71	6	7
## px_151	-0.03	0.21	-0.54	0.21 4.77	6	11
## px_152	0.00	0.22	-0.25	0.48 5.82	6	7
## px_153	0.03	0.20	-0.33	0.29 6.65	6	11
## px_154	-0.02	0.21	-0.31	0.51 5.86	6	7
## px_155	0.04	0.12	-0.10	0.20 7.06	6	11
## px_156	0.02	0.09	-0.14	0.18 5.18	6	7

## px_157	-0.11	0.09	-0.28	0.03 3.47	7	11
## px_158	0.14	0.09	0.01	0.32 3.77	6	7
## px_159	-0.07	0.11	-0.19	0.21 5.00	6	6
## px_160	-0.02	0.07	-0.18	0.08 5.79	6	NA
## px_161	-0.10	0.11	-0.30	0.05 5.46	6	6
## px_162	0.14	0.15	-0.07	0.37 5.10	6	6
## px_163	-0.08	0.12	-0.26	0.10 4.77	6	6
## px_164	0.01	0.12	-0.31	0.11 5.33	6	11
## px_165	-0.12	0.13	-0.32	0.06 7.06	6	6
## px_166	0.13	0.11	-0.08	0.32 3.84	6	11
## px_167	-0.07	0.14	-0.32	0.11 5.12	6	6
## px_168	0.08	0.11	-0.08	0.26 4.79	6	11
## px_169	-0.07	0.12	-0.32	0.08 4.26	6	6
## px_170	0.06	0.15	-0.14	0.36 5.75	6	11
## px_171	-0.05	0.13	-0.28	0.08 4.34	6	6
## px_172	-0.03	0.08	-0.17	0.14 6.21	6	11
## px_173	0.11	0.08	-0.02	0.27 5.04	6	11
## px_174	-0.10	0.06	-0.21	-0.00 3.48	7	15
## px_175	0.07	0.10	-0.10	0.24 5.35	6	11
## px_176	-0.00	0.04	-0.05	0.08 3.82	7	7
## px_177	0.03	0.11	-0.19	0.16 5.46	6	11
## px_178	-0.03	0.14	-0.24	0.24 4.37	6	11
## px_179	-0.02	0.10	-0.20	0.13 4.49	6	6
## px_180	0.03	0.08	-0.08	0.17 4.82	6	7
## px_181	0.08	0.05	0.01	0.19 2.88	7	7
## px_182	-0.09	0.08	-0.27	0.03 3.66	6	11
## px_183	0.05	0.07	-0.05	0.18 5.73	6	6
## px_184	-0.07	0.11	-0.26	0.15 3.69	6	11
## px_185	0.05	0.06	-0.07	0.14 4.39	6	7
## px_186	-0.05	0.09	-0.23	0.06 4.93	6	6
## px_187	0.06	0.13	-0.08	0.38 3.42	7	11
## px_188	-0.02	0.16	-0.26	0.25 5.92	6	11
## px_189	-0.08	0.12	-0.27	0.07 5.23	6	6
## px_190	0.03	0.06	-0.09	0.10 5.04	6	6
## px_191	-0.01	0.10	-0.17	0.18 6.23	6	6
## px_192	-0.05	0.06	-0.15	0.05 4.67	6	6
## px_193	-0.05	0.15	-0.30	0.12 4.63	6	6
## px_194	0.02	0.16	-0.20	0.24 3.58	7	11
## px_195	0.01	0.10	-0.17	0.19 6.91	6	NA
## px_196	-0.02	0.12	-0.24	0.14 5.46	6	6
## px_197	-0.05	0.05	-0.17	0.01 5.10	7	7
## px_198	0.08	0.09	-0.06	0.22 5.45	6	11
## px_199	-0.07	0.14	-0.28	0.21 5.20	6	11
## px_200	0.01	0.11	-0.10	0.22 5.87	6	7
## px_201	-0.01	0.07	-0.11	0.15 4.02	6	11
## px_202	0.05	0.11	-0.18	0.22 5.03	6	11
## px_203	-0.09	0.13	-0.33	0.07 3.77	6	11
## px_204	0.05	0.09	-0.10	0.21 5.31	6	11
## px_205	0.06	0.09	-0.04	0.27 5.85	6	11
## px_206	0.01	0.15	-0.16	0.31 5.38	6	11
## px_207	-0.05	0.08	-0.21	0.04 4.52	6	11
## px_208	0.09	0.11	-0.03	0.30 4.59	6	6
## px_209	0.02	0.06	-0.06	0.13 4.29	6	11
## px_210	-0.04	0.08	-0.20	0.09 4.57	6	14

## px_211	0.01	0.10	-0.08	0.24 5.32	6	6
## px_212	0.00	0.13	-0.23	0.21 5.24	6	11
## px_213	0.04	0.08	-0.10	0.18 5.62	6	6
## px_214	-0.06	0.10	-0.22	0.11 5.48	6	11
## px_215	0.09	0.13	-0.13	0.26 5.27	6	6
## px_216	-0.01	0.13	-0.17	0.23 5.18	6	11
## px_217	-0.04	0.07	-0.17	0.08 5.84	6	7
## px_218	-0.00	0.12	-0.18	0.18 6.11	6	6
## px_219	0.09	0.21	-0.18	0.48 5.54	6	7
## px_220	-0.11	0.21	-0.54	0.07 4.38	6	6
## px_221	0.02	0.23	-0.21	0.51 4.90	6	12
## px_222	-0.06	0.27	-0.65	0.14 4.66	6	6
## px_223	0.06	0.14	-0.16	0.32 6.33	6	6
## px_224	-0.11	0.15	-0.41	0.05 5.17	6	6
## px_225	-0.02	0.13	-0.29	0.12 7.00	6	6
## px_226	0.07	0.19	-0.15	0.46 4.72	6	6
## px_227	-0.06	0.18	-0.39	0.16 4.66	6	6
## px_228	0.05	0.23	-0.20	0.49 6.63	6	6
## px_229	-0.08	0.21	-0.48	0.15 5.29	6	6
## px_230	0.04	0.22	-0.21	0.44 5.26	6	7
## px_231	-0.05	0.20	-0.38	0.16 5.69	6	6
## px_232	0.03	0.17	-0.19	0.29 5.14	6	17
## px_233	0.07	0.10	-0.11	0.23 4.99	6	11
## px_234	-0.04	0.12	-0.30	0.07 6.87	6	NA
## px_235	-0.04	0.20	-0.41	0.29 6.68	6	6
## px_236	0.07	0.22	-0.21	0.51 5.42	6	7
## px_237	0.01	0.19	-0.38	0.22 5.10	6	6
## px_238	0.02	0.22	-0.17	0.50 3.84	6	23
## px_239	0.02	0.09	-0.13	0.14 6.08	6	6
## px_240	0.05	0.09	-0.09	0.21 5.56	6	NA
## px_241	0.00	0.14	-0.16	0.28 4.37	6	11
## px_242	-0.05	0.18	-0.41	0.15 3.82	6	6
## px_243	0.02	0.13	-0.20	0.25 4.12	6	11
## px_244	-0.04	0.16	-0.37	0.11 7.52	6	6
## px_245	0.05	0.18	-0.14	0.40 5.47	6	7
## px_246	-0.06	0.22	-0.45	0.20 5.38	6	6
## px_247	0.00	0.15	-0.19	0.23 3.93	6	11
## px_248	0.01	0.13	-0.16	0.20 3.97	6	7
## px_249	-0.06	0.08	-0.24	0.07 4.96	6	7
## px_250	-0.02	0.14	-0.20	0.30 5.52	6	11
## px_251	0.05	0.20	-0.32	0.41 4.50	6	7
## px_252	-0.09	0.24	-0.50	0.23 3.95	6	7
## px_253	0.06	0.27	-0.18	0.53 4.54	6	22
## px_254	-0.12	0.30	-0.68	0.17 4.84	6	6
## px_255	0.09	0.21	-0.13	0.46 6.51	6	6
## px_256	-0.13	0.15	-0.42	0.02 6.12	6	6
##						
##	Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS					
##	and Tail_ESS are effective sample size measures, and Rhat is the potential					
##	scale reduction factor on split chains (at convergence, Rhat = 1).					

```
post_draws <- as.data.frame(brm_logit)
```

Model validation

```
library(caret)
```

```
## Loading required package: lattice
```

```
##
```

```
## Attaching package: 'caret'
```

```
## The following object is masked from 'package:purrr':
```

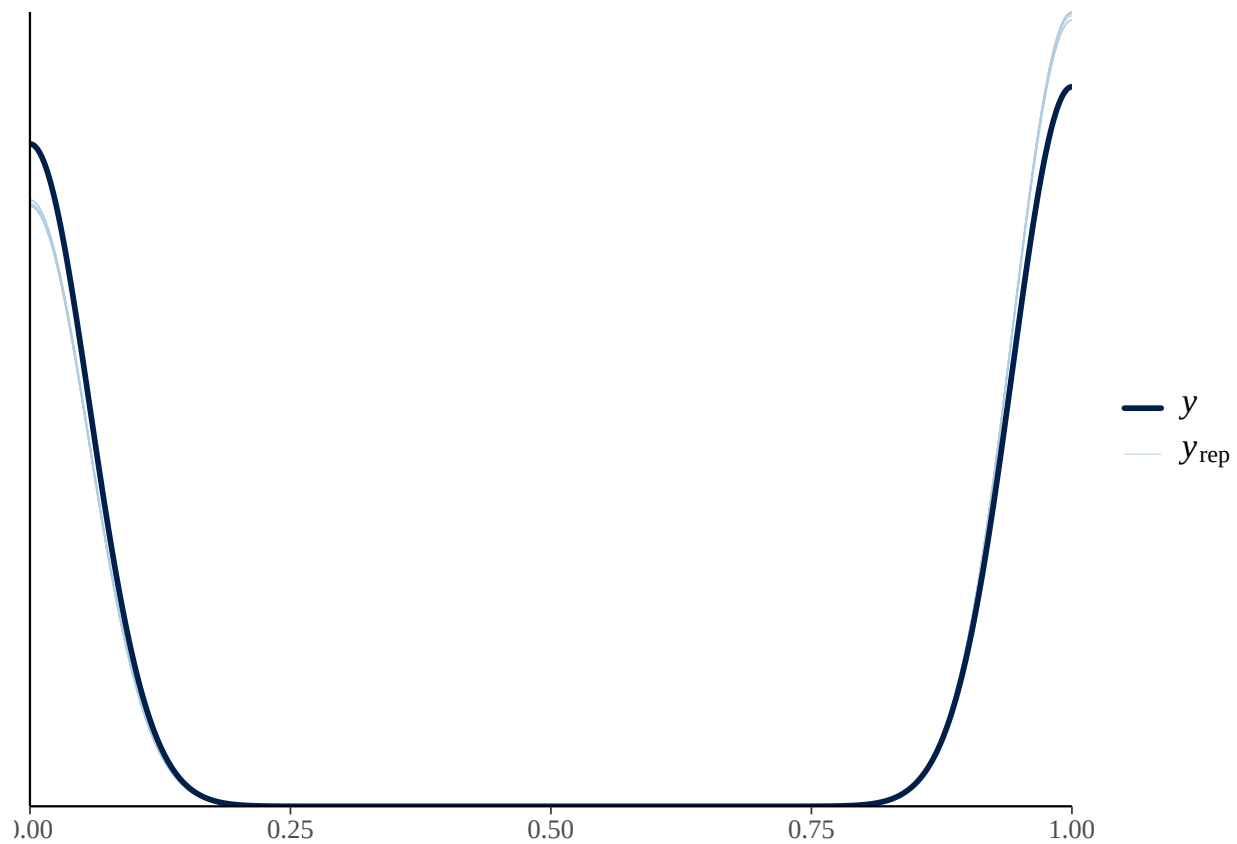
```
##
```

```
## lift
```

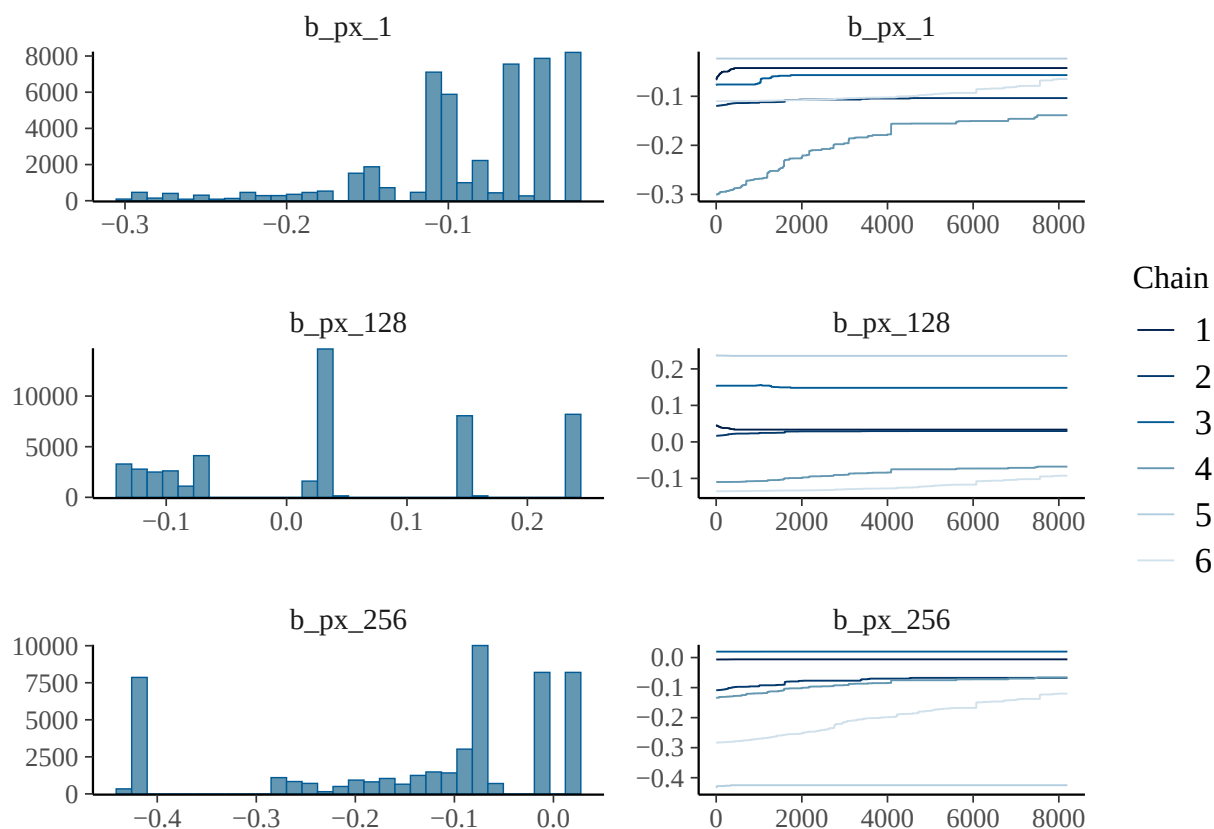
```
# Check posterior predictive fit
```

```
pp_check(brm_logit)
```

```
## Using 10 posterior draws for ppc type 'dens_overlay' by default.
```




```
# Plot individual features
plot(brm_logit, variable = c("b_px_1", "b_px_128", "b_px_256"))
```



```
# Evaluate on validation set
pred_probs <- posterior_epred(brm_logit, newdata = valid_df)
pred_label <- as.numeric(colMeans(pred_probs) > 0.5)
```

```
# Confusion matrix
confusionMatrix(factor(pred_label, levels=c(1,0)),
                 factor(valid_df$label, levels=c(1,0)),
                 mode = "everything")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    1    0
##           1 2903  520
##           0  577 2300
##
##           Accuracy : 0.8259
##           95% CI : (0.8163, 0.8352)
##           No Information Rate : 0.5524
##           P-Value [Acc > NIR] : < 2e-16
##
##           Kappa : 0.6486
```

```
##
## Mcnemar's Test P-Value : 0.09088
##
##          Sensitivity : 0.8342
##          Specificity : 0.8156
##          Pos Pred Value : 0.8481
##          Neg Pred Value : 0.7994
##          Precision : 0.8481
##          Recall : 0.8342
##          F1 : 0.8411
##          Prevalence : 0.5524
##          Detection Rate : 0.4608
##          Detection Prevalence : 0.5433
##          Balanced Accuracy : 0.8249
##
##          'Positive' Class : 1
##
```

Model testing

```
# Evaluate on test set
pred_probs <- posterior_epred(brm_logit, newdata = test_df)
pred_label <- as.numeric(colMeans(pred_probs) > 0.5)

# Confusion matrix, get F1, precision, recall
test_conf_matrix <- confusionMatrix(factor(pred_label, levels=c(1,0)),
                                     factor(test_df$label, levels=c(1,0)),
                                     mode = "everything")
test_conf_matrix
```

```
## Confusion Matrix and Statistics
##
##          Reference
## Prediction    1    0
##          1 2877  531
##          0  602 2289
##
##          Accuracy : 0.8201
##          95% CI : (0.8104, 0.8295)
##          No Information Rate : 0.5523
##          P-Value [Acc > NIR] : < 2e-16
##
##          Kappa : 0.6371
##
## Mcnemar's Test P-Value : 0.03756
##
##          Sensitivity : 0.8270
##          Specificity : 0.8117
##          Pos Pred Value : 0.8442
##          Neg Pred Value : 0.7918
##          Precision : 0.8442
##          Recall : 0.8270
```

```
##                F1 : 0.8355
##            Prevalence : 0.5523
##        Detection Rate : 0.4567
##    Detection Prevalence : 0.5410
##        Balanced Accuracy : 0.8193
##
##        'Positive' Class : 1
##
```

```
# Get per-class accuracies
test_conf_mat_2 <- table(Predicted = pred_label, Actual = test_df$label)
true_positives <- diag(test_conf_mat_2) # TP
actual_counts <- colSums(test_conf_mat_2) # Total per class
per_class_accuracy <- true_positives / actual_counts # Per-class acc
print(per_class_accuracy)
```

```
##          0          1
## 0.8117021 0.8269618
```

```
# Generate confusion matrix figure
library(ggplot2)
plt <- as.data.frame(test_conf_matrix$table)
ggplot(plt, aes(Prediction, Reference, fill = Freq)) +
  geom_tile() +
  geom_text(aes(label = Freq)) +
  scale_fill_gradient(low = "steelblue", high = "white") +
  ggtitle("Bayesian LR Confusion Matrix") +
  theme_minimal()
```

